

BugBoard26

Documento di design del sistema

Progetto di Ingegneria del Software
A.A. 2025/2026

Componenti	Valerio Porcile, Emanuel Giuseppe Lupoli, Emanuele Martucci
Modalità	Tradizionale
Versione	1.0
Data	25 giugno 2026

Indice

1	Introduzione	2
2	Obiettivi di progettazione	2
3	Architettura generale	3
3.1	Component diagram	3
3.2	Deployment diagram	4
4	Scelte tecnologiche	5
4.1	Frontend Angular	5
4.2	Backend ASP.NET Core Web API	5
4.3	Database PostgreSQL	5
4.4	Autenticazione JWT	5
5	Organizzazione del backend	6
5.1	Controller	6
5.2	Servizi applicativi	6
5.3	Repository	6
5.4	DbContext	6
6	Class diagram di design	7
6.1	Vista dominio e persistenza	7
6.2	Vista servizi e repository	8
7	Schema della persistenza dati	9
8	Principali scelte di design	10
8.1	Separazione frontend e backend	10
8.2	Uso di servizi applicativi	10
8.3	Uso di repository	10
8.4	Gestione dei ruoli	10
8.5	Suggerimento automatico dello assegnatario	10
8.6	Gestione delle notifiche	10
8.7	Gestione dei duplicati	11
8.8	Archiviazione	11
9	Diagrammi comportamentali	11
9.1	Creazione di una issue	11
9.2	Cambio stato con notifica	12
9.3	Contrassegno di una issue duplicata	12
10	API principali previste	13
11	Testabilità	14
12	Conclusioni	14

1 Introduzione

Il presente documento descrive la progettazione del sistema BugBoard26, realizzato come applicazione distribuita per la gestione collaborativa di issue in progetti software.

Il documento rappresenta la seconda fase del lavoro progettuale e riprende quanto definito nella specifica dei requisiti. In particolare, vengono descritte le scelte architeturali, la suddivisione delle responsabilità tra frontend, backend e database, la struttura principale delle classi di design e le motivazioni delle principali scelte tecnologiche.

Le funzionalità considerate sono quelle assegnate al gruppo:

- autenticazione tramite email e password;
- creazione di utenti da parte degli amministratori;
- creazione di issue da parte degli utenti autenticati;
- visualizzazione, ricerca, filtro e ordinamento delle issue;
- cambio stato delle issue assegnate;
- notifica al segnalatore quando una issue viene risolta;
- esportazione delle issue in formato CSV;
- archiviazione delle issue da parte degli amministratori;
- suggerimento automatico dell'assegnatario in base al carico di lavoro;
- gestione di utenti readonly;
- contrassegno di una issue come duplicata.

2 Obiettivi di progettazione

La progettazione del sistema è stata guidata dai seguenti obiettivi:

- separare chiaramente frontend, backend e persistenza;
- mantenere il backend indipendente dalla tecnologia utilizzata per la interfaccia utente;
- esporre le funzionalità tramite API REST;
- centralizzare la logica applicativa nel backend;
- applicare una struttura a livelli, distinguendo controller, servizi, repository e persistenza;
- limitare la complessità, mantenendo il sistema coerente con il lavoro di un gruppo di tre studenti;
- rendere il progetto facilmente testabile, estendibile e spiegabile in sede di discussione.

3 Architettura generale

Il sistema è stato progettato come applicazione web distribuita composta da tre macro componenti:

- **Frontend Angular**, responsabile della interfaccia utente e della interazione con lo utente;
- **Backend ASP.NET Core Web API**, responsabile della logica applicativa, della autenticazione, della gestione dei ruoli e della esposizione delle API REST;
- **Database PostgreSQL**, responsabile della persistenza dei dati.

La comunicazione tra frontend e backend avviene esclusivamente tramite richieste HTTP verso API REST. Le richieste protette includono un token JWT generato dal backend durante la fase di login.

Questa scelta permette di rispettare il vincolo di separazione tra frontend e backend: il backend non dipende da Angular e potrebbe essere utilizzato anche da una diversa interfaccia, per esempio una applicazione mobile o desktop.

3.1 Component diagram

Il component diagram mostra la suddivisione logica del sistema. Il frontend contiene i componenti grafici e i servizi Angular che invocano le API del backend. Il backend è organizzato in controller, servizi applicativi, repository e DbContext. Il database contiene le tabelle principali del sistema.

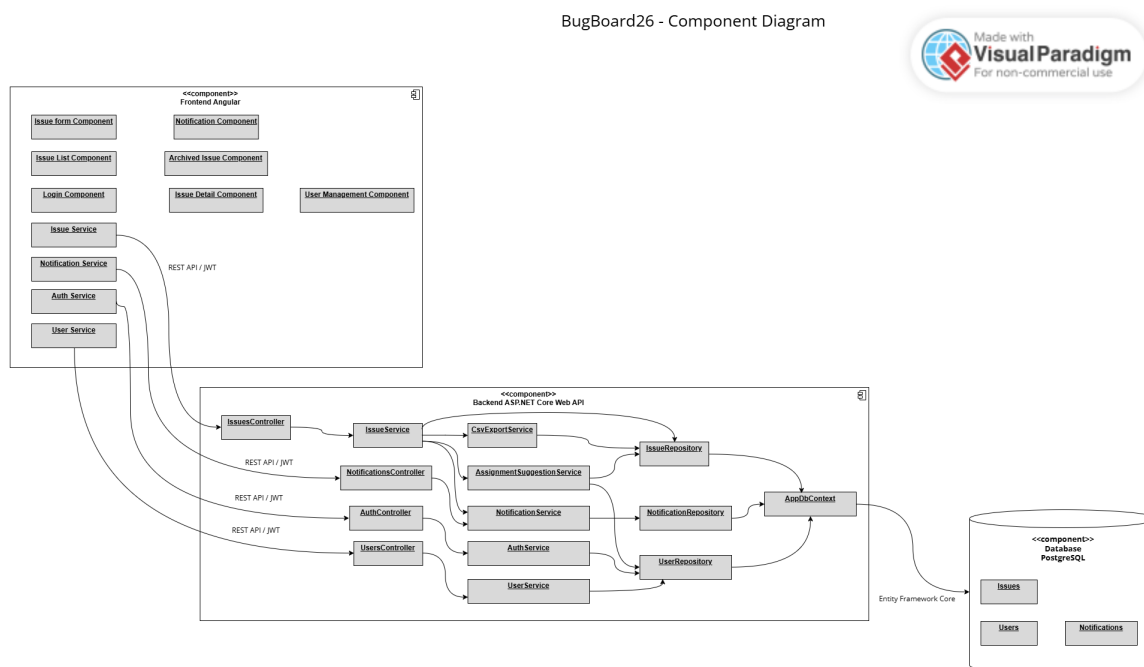


Figura 1: Component diagram di BugBoard26

Nel diagramma sono evidenziate le principali dipendenze tra i componenti: il frontend comunica con i controller del backend tramite API REST e JWT, i controller delegano la logica ai servizi, i servizi utilizzano repository e il livello di persistenza accede al database tramite Entity Framework Core.

3.2 Deployment diagram

Il deployment diagram mostra una possibile distribuzione del sistema. Lo utente accede alla applicazione tramite browser. La parte frontend comunica con il backend tramite rete, mentre il backend accede al database PostgreSQL tramite protocollo PostgreSQL.

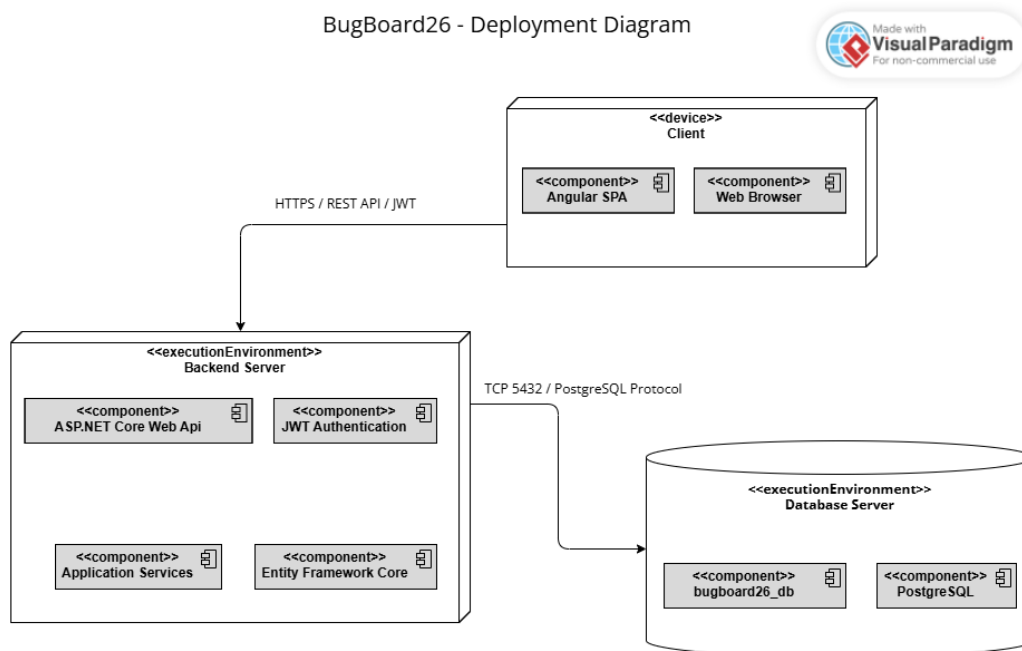


Figura 2: Deployment diagram di BugBoard26

La separazione in nodi distinti consente di eseguire le componenti in modo indipendente. In ambiente di sviluppo il database viene eseguito tramite Docker, mentre frontend e backend possono essere avviati separatamente. In una futura distribuzione, anche il backend e il frontend potrebbero essere eseguiti in container dedicati.

4 Scelte tecnologiche

4.1 Frontend Angular

Angular è stato scelto per realizzare una Single Page Application strutturata in componenti. Questa tecnologia permette di separare la presentazione dalla logica di accesso alle API e offre strumenti già integrati per routing, form, servizi e gestione delle chiamate HTTP.

Nel frontend sono previsti componenti dedicati alle principali schermate:

- login;
- lista issue;
- creazione issue;
- dettaglio issue;
- archivio issue;
- notifiche;
- gestione utenti per amministratori.

4.2 Backend ASP.NET Core Web API

ASP.NET Core Web API è stato scelto per il backend perché supporta in modo nativo la creazione di API REST, la gestione della dependency injection, la autenticazione JWT e la integrazione con Entity Framework Core.

Il backend è organizzato secondo una struttura a livelli:

- **Controller**: espongono gli endpoint REST;
- **Service**: contengono la logica applicativa;
- **Repository**: isolano lo accesso ai dati;
- **DbContext**: rappresenta il collegamento con il database.

Questa struttura consente di separare le responsabilità e rende più semplice testare la logica applicativa.

4.3 Database PostgreSQL

PostgreSQL è stato scelto come database relazionale per la persistenza delle informazioni principali del sistema. Le entità principali sono utenti, issue e notifiche.

La scelta di un database relazionale è coerente con la natura dei dati, che presentano relazioni chiare tra utenti, issue e notifiche.

4.4 Autenticazione JWT

La autenticazione è basata su token JWT. Dopo il login, il backend restituisce un token contenente le informazioni essenziali dello utente, tra cui identificativo e ruolo. Il frontend invia il token nelle richieste successive verso le API protette.

Questa soluzione permette di mantenere il backend stateless rispetto alle sessioni e di gestire in modo semplice autorizzazioni basate sui ruoli.

5 Organizzazione del backend

Il backend è stato progettato separando le responsabilità in più livelli.

5.1 Controller

I controller rappresentano il punto di ingresso delle richieste HTTP. Ogni controller espone un gruppo di API legate a una specifica area funzionale:

- `AuthController`: login e generazione del token JWT;
- `UsersController`: creazione e consultazione degli utenti;
- `IssuesController`: creazione, ricerca, filtro, cambio stato, archiviazione, duplicati ed export;
- `NotificationsController`: consultazione e lettura delle notifiche.

5.2 Servizi applicativi

I servizi contengono la logica del sistema. In questo modo i controller rimangono semplici e si limitano a ricevere le richieste e restituire le risposte.

I principali servizi previsti sono:

- `AuthService`;
- `UserService`;
- `IssueService`;
- `NotificationService`;
- `AssignmentSuggestionService`;
- `CsvExportService`.

5.3 Repository

I repository isolano la logica di accesso ai dati. I servizi dipendono dalle interfacce dei repository e non direttamente dalla implementazione concreta. Questa scelta rende il sistema più testabile, poiché nei test è possibile sostituire i repository con implementazioni fittizie.

5.4 DbContext

`AppDbContext` rappresenta il punto di integrazione con Entity Framework Core. Al suo interno sono dichiarati i `DbSet` principali:

- `Users`;
- `Issues`;
- `Notifications`.

6 Class diagram di design

Per migliorare la leggibilità, il class diagram è stato suddiviso in due viste complementari. La prima descrive il dominio e la persistenza, mentre la seconda mostra servizi, repository e dipendenze principali.

6.1 Vista dominio e persistenza

La vista dominio e persistenza mostra le entità principali del sistema e le relazioni tra esse.

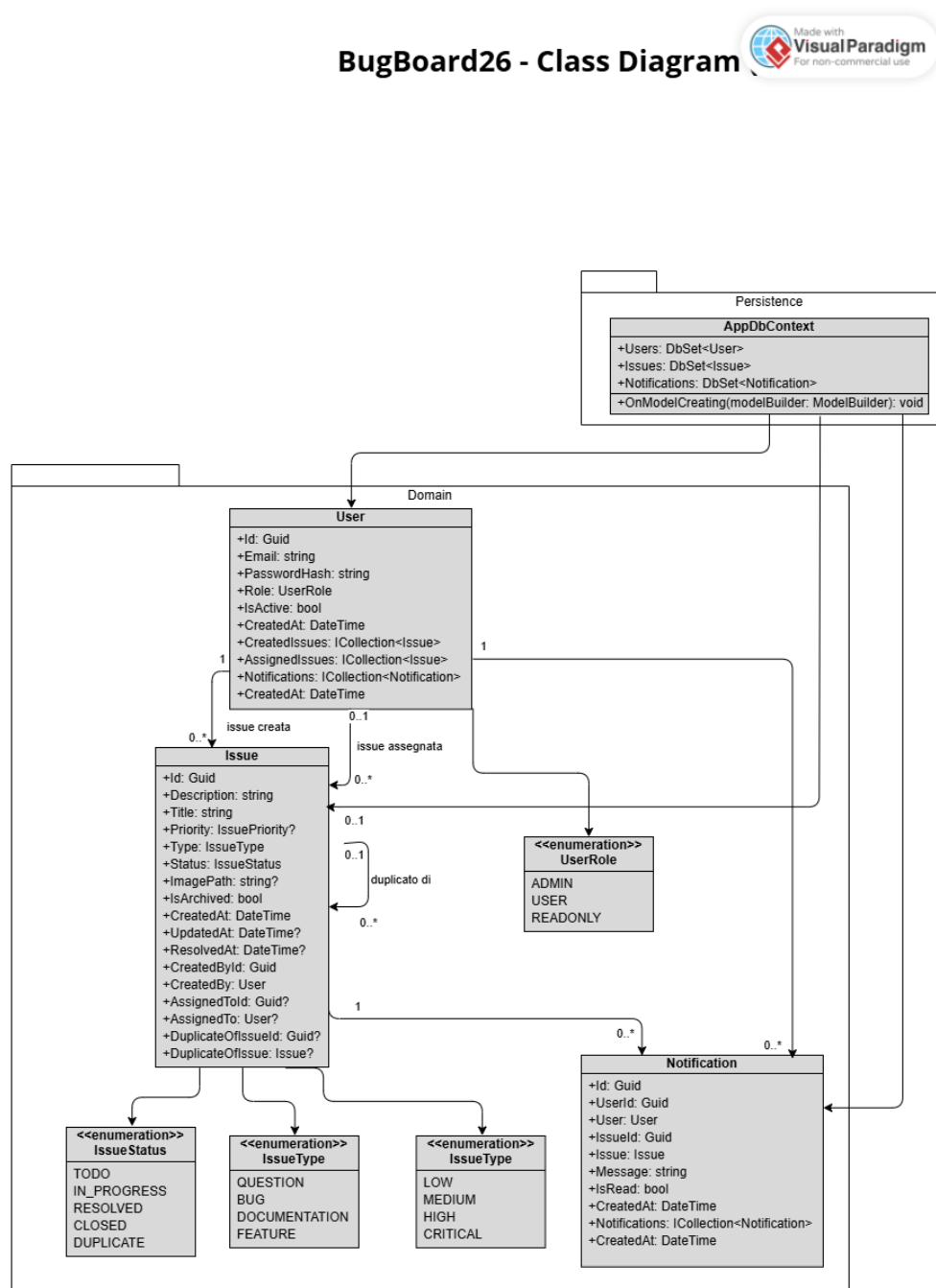


Figura 3: Class diagram: dominio e persistenza

Le entità principali sono User, Issue e Notification. Una issue è creata da uno utente e può essere assegnata a uno utente. Una issue può inoltre essere marcata come duplicata di un'altra issue. Le notifiche sono associate sia allo utente destinatario sia alla issue di riferimento.

6.2 Vista servizi e repository

La vista servizi e repository mostra la organizzazione della logica applicativa del backend.

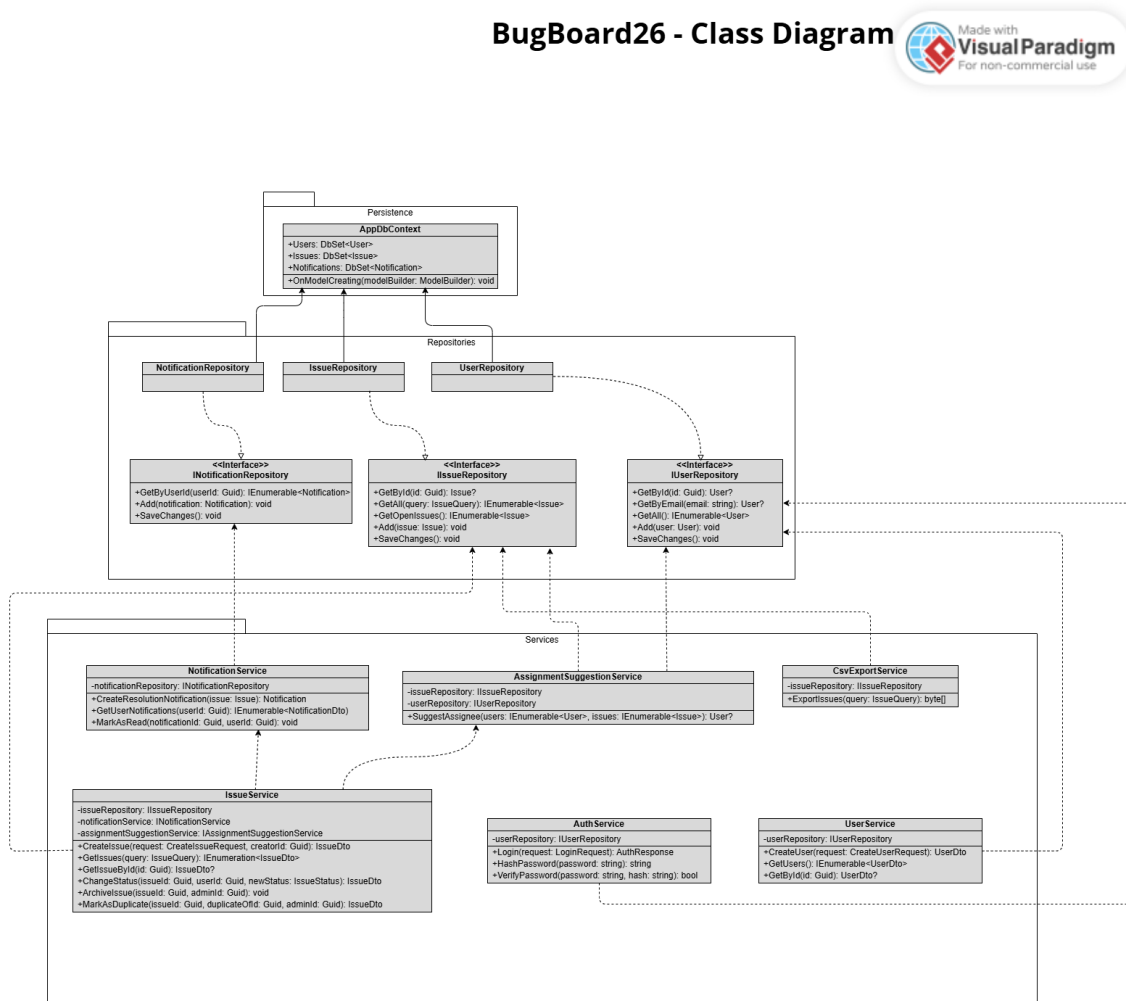


Figura 4: Class diagram: servizi e repository

I servizi applicativi utilizzano le interfacce dei repository per accedere ai dati. Le classi concrete dei repository implementano tali interfacce e usano `AppDbContext`. Questa struttura riduce lo accoppiamento tra logica applicativa e persistenza.

7 Schema della persistenza dati

La persistenza dei dati è basata su tre tabelle principali:

- Users;
- Issues;
- Notifications.

BugBoard26 - Database Schema

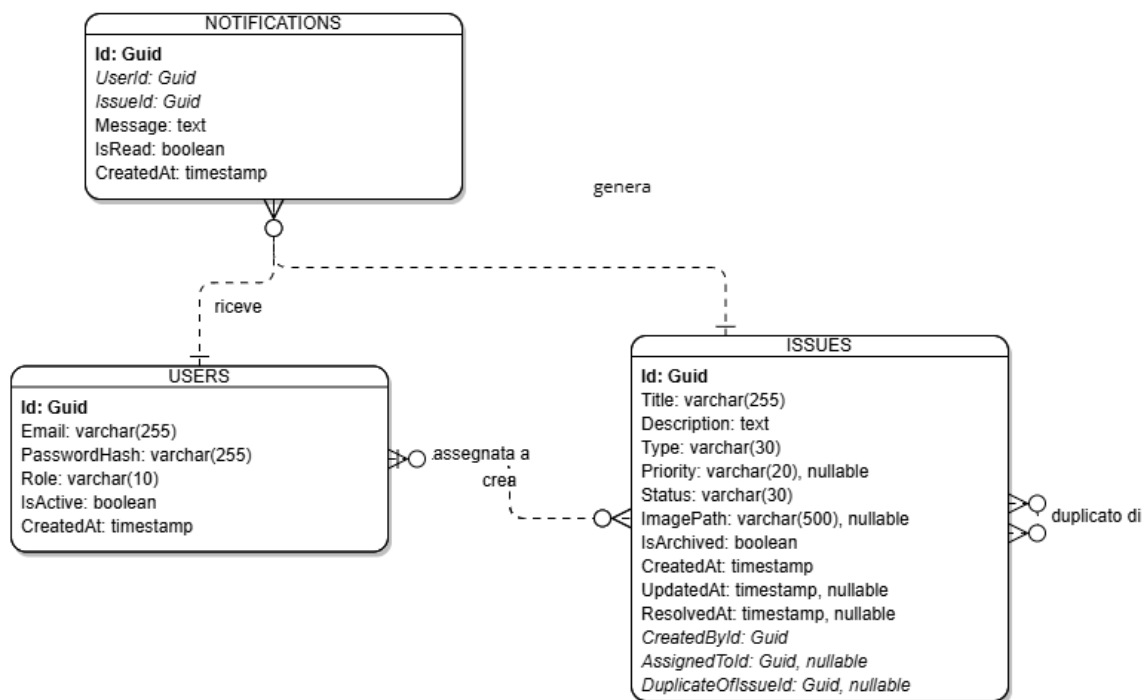


Figura 5: Schema della persistenza dati

Tabella	Descrizione
Users	Contiene gli utenti del sistema, le credenziali cifrate tramite hash, il ruolo e lo stato di attivazione.
Issues	Contiene le issue create dagli utenti, il loro stato, la priorità, il tipo, lo assegnatario, le informazioni di archiviazione e lo eventuale collegamento a una issue duplicata.
Notifications	Contiene le notifiche generate dal sistema, in particolare quelle relative alla risoluzione di una issue.

8 Principali scelte di design

8.1 Separazione frontend e backend

Il frontend non accede direttamente al database e non contiene logica di business critica. Tutte le regole principali, come controllo dei ruoli, cambio stato, archiviazione e gestione dei duplicati, sono implementate nel backend.

8.2 Uso di servizi applicativi

La logica applicativa è concentrata nei servizi. Questa scelta permette di mantenere i controller più semplici e di testare più facilmente le operazioni principali.

8.3 Uso di repository

I repository sono stati introdotti per separare la logica applicativa dallo accesso ai dati. I servizi dipendono da interfacce, non da implementazioni concrete. Questo favorisce testabilità e sostituibilità delle implementazioni.

8.4 Gestione dei ruoli

Il sistema prevede tre ruoli principali:

- **ADMIN:** può gestire utenti, archiviare issue, esportare dati e contrassegnare duplicati;
- **USER:** può creare issue, consultarle e cambiare lo stato delle issue assegnate;
- **READONLY:** può solo consultare le issue, senza modificarle.

Il controllo dei ruoli viene effettuato lato backend, in modo da impedire operazioni non autorizzate anche in caso di manipolazione della interfaccia frontend.

8.5 Suggerimento automatico dello assegnatario

Il suggerimento automatico dello assegnatario è realizzato considerando il carico di lavoro corrente degli utenti. Il criterio di base è selezionare lo utente normale con il minor numero di issue aperte assegnate.

Le issue risolte, duplicate o archiviate non vengono considerate come carico attivo. In caso di parità è possibile scegliere un criterio stabile, per esempio ordinamento per email.

8.6 Gestione delle notifiche

Quando una issue viene portata allo stato **RESOLVED**, il sistema genera una notifica per lo utente che ha creato la issue. La notifica viene salvata nel database e può essere visualizzata dalla interfaccia utente.

8.7 Gestione dei duplicati

Solo gli amministratori possono contrassegnare una issue come duplicata. Quando questo avviene, la issue viene collegata a una issue principale e il suo stato viene impostato a **DUPLICATE**. In questo modo rimane tracciabile ma non viene più considerata come issue aperta da risolvere.

8.8 Archiviazione

Solo gli amministratori possono archiviare una issue. Una issue archiviata non viene mostrata nella lista principale, ma resta accessibile da una sezione dedicata. Questa scelta permette di non eliminare dati utili e di conservare la tracciabilità delle segnalazioni.

9 Diagrammi comportamentali

I diagrammi comportamentali descrivono il flusso delle interazioni tra frontend, controller, servizi, repository e database per alcune funzionalità centrali del sistema. Sono stati scelti i casi relativi alla creazione di una issue, al cambio stato con generazione della notifica e alla gestione dei duplicati, perché rappresentano parti significative della logica applicativa.

9.1 Creazione di una issue

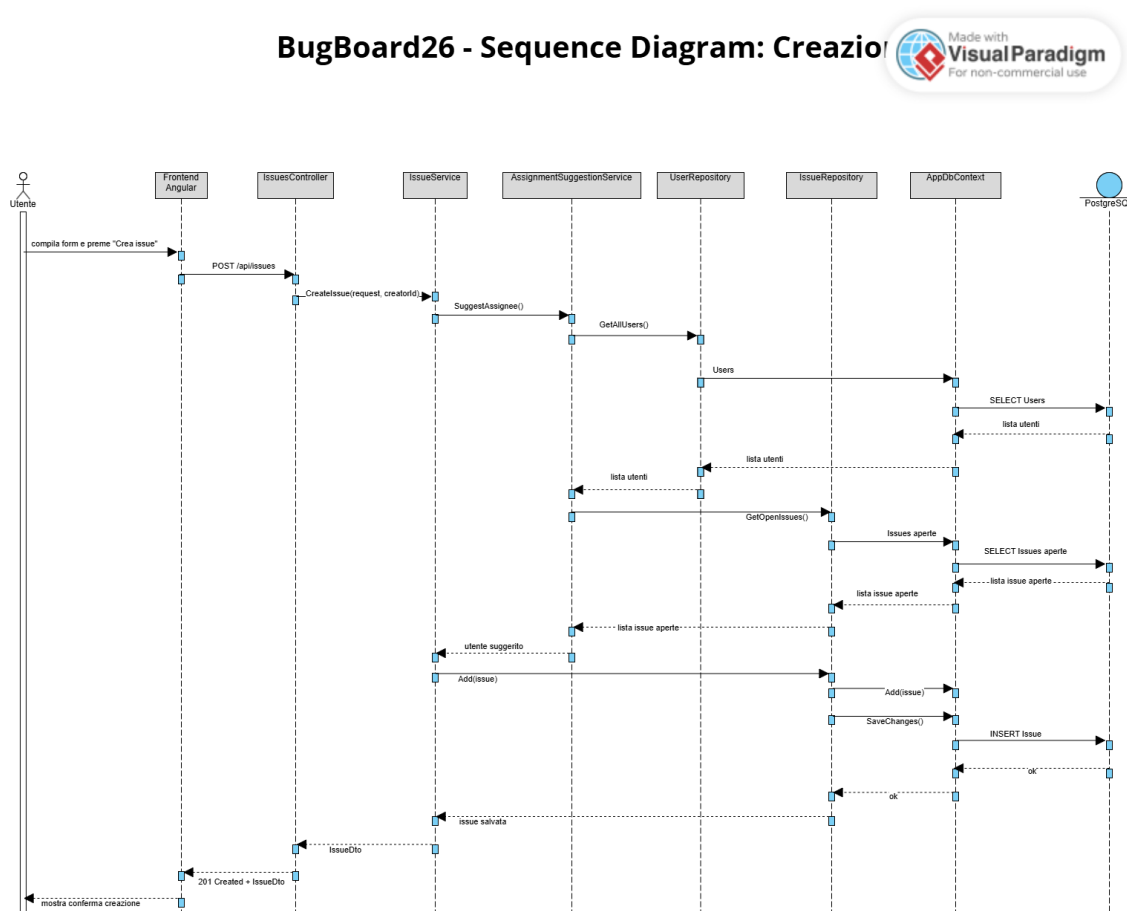


Figura 6: Sequence diagram: creazione di una issue

9.2 Cambio stato con notifica

BugBoard26 - Sequence diagram: Cambio stato con notif

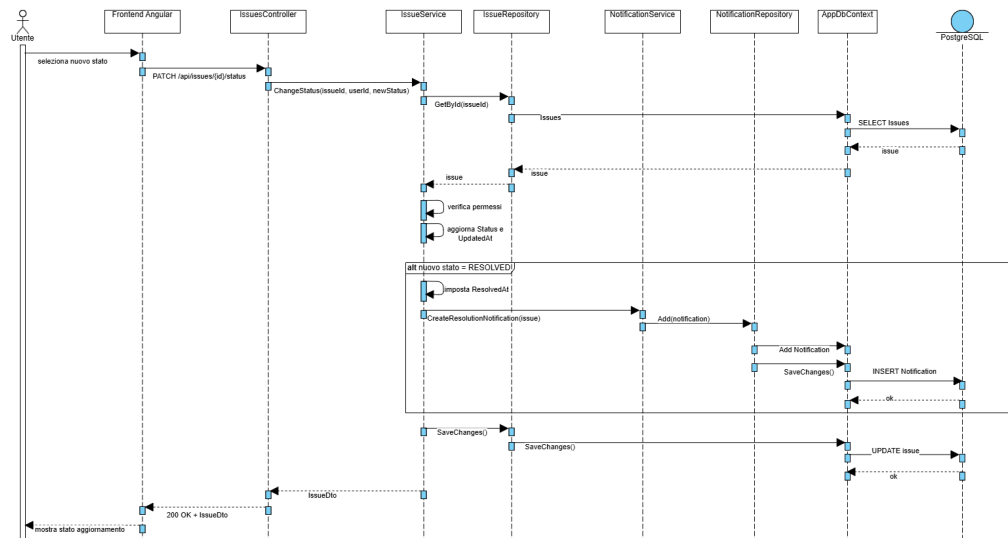


Figura 7: Sequence diagram: cambio stato con notifica

9.3 Contrassegno di una issue duplicata

BugBoard26 - Sequence Diagram: contrassegno issue

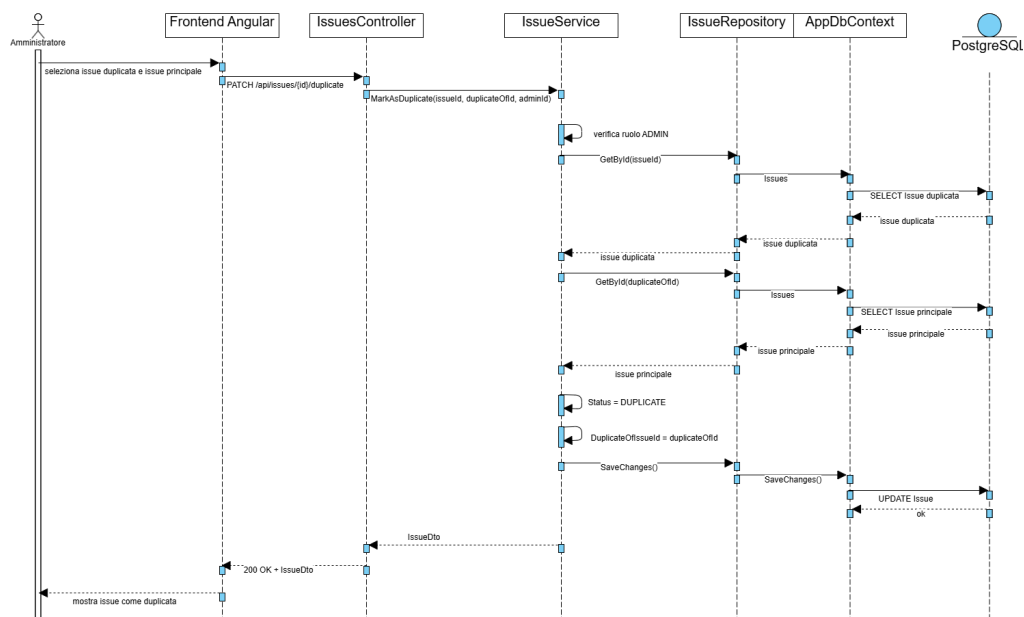


Figura 8: Sequence diagram: contrassegno di una issue duplicata

10 API principali previste

Metodo	Endpoint	Descrizione
POST	/api/auth/login	Effettua il login e restituisce il token JWT.
POST	/api/users	Crea un nuovo utente. Operazione riservata agli amministratori.
GET	/api/users	Restituisce la lista degli utenti.
POST	/api/issues	Crea una nuova issue.
GET	/api/issues	Restituisce le issue con ricerca, filtri e ordinamento.
GET	/api/issues/{id}	Restituisce il dettaglio di una issue.
PATCH	/api/issues/{id}/status	Modifica lo stato di una issue assegnata.
PATCH	/api/issues/{id}/archive	Archivia una issue. Operazione riservata agli amministratori.
GET	/api/issues/archived	Restituisce le issue archiviate.
PATCH	/api/issues/{id}/duplicate	Contrassegna una issue come duplicata.
GET	/api/issues/export	Esporta le issue in formato CSV.
GET	/api/notifications	Restituisce le notifiche dell'utente autenticato.
PATCH	/api/notifications/{id}/read	Segna una notifica come letta.

Tabella 2: Endpoint principali esposti dal backend

11 Testabilità

La progettazione è stata impostata per rendere testabili le parti principali della logica applicativa. In particolare, i servizi non dipendono direttamente dal database, ma dalle interfacce dei repository. Questo consente di testare i servizi usando dati controllati.

Le funzionalità più adatte ai test automatici sono:

- suggerimento dello assegnatario;
- verifica dei permessi per il cambio stato;
- ricerca e filtro delle issue;
- generazione della notifica alla risoluzione;
- gestione del duplicato.

12 Conclusioni

La progettazione proposta divide BugBoard26 in componenti indipendenti e con responsabilità chiare. Il frontend Angular si occupa della presentazione, il backend ASP.NET Core contiene la logica applicativa e PostgreSQL gestisce la persistenza dei dati.

La struttura a livelli adottata rende il sistema più ordinato, testabile ed estendibile. Le scelte di design sono coerenti con le funzionalità assegnate e con il vincolo di realizzare un sistema distribuito con frontend e backend separati.